

Contents

1	Introduction	2
1.1	Objectifs du rapport	2
1.2	Méthodes de Clustering : Une Vue d'Ensemble	2
1.3	Méthodes de Clustering dans l'Analyse de Données	2
2	Algorithme de K-Means	3
2.1	Définition et Origines du K-Means	3
2.2	Principes Mathématiques du K-Means	3
2.3	Fonctionnement de l'Algorithme K-Means	4
2.3.1	Illustration de la Méthode:	5
2.3.2	Évaluation de la Performance de l'Algorithme K-Means:	5
2.4	Avantages et Limites	6
2.5	Implémentation du K-Means en Python	6
2.5.1	Environnement de Développement:	6
2.5.2	Développement de l'algorithme : Etapes et Code de base :	7
2.5.3	Méthode du Coude pour le Choix de k	10
3	Classification Hiérarchique	11
3.1	Introduction à la Classification Hiérarchique	11
3.2	Mécanique de la Classification Hiérarchique	11
3.2.1	Méthodes de liaisons :	11
3.3	Calcul des Distances et Similarités	13
3.4	Critères de Fusion dans la Classification Hiérarchique	13
3.5	Implémentation de CAH en Python	14
3.5.1	Choix de l'environnement pour CAH :	14
4	Résultats : Application Web de Clustering	18
4.1	Code Source de l'Application	18
4.2	L'Application Web	26
4.3	Visualisation de l'Application Web	26
	Conclusion	28

1 Introduction

1.1 Objectifs du rapport

Le présent rapport propose une analyse détaillée et la mise en œuvre de deux techniques fondamentales de regroupement de données : l'algorithme K-means et la classification hiérarchique, en utilisant le langage de programmation Python. Ces méthodes jouent un rôle essentiel dans l'analyse de données en permettant de regrouper un ensemble d'observations en sous-ensembles homogènes, où les objets au sein d'un même cluster présentent une plus grande similarité entre eux qu'avec ceux des autres clusters. Le premier partie est consacré à l'étude de l'algorithme K-means, en abordant ses fondements théoriques, son processus d'optimisation, son implémentation en Python, ainsi que l'analyse d'un cas pratique illustrant son efficacité. La deuxième partie elle traite la classification hiérarchique, en présentant ses principes, ses différentes variantes, sa mise en œuvre en Python, ainsi que l'interprétation de résultats obtenus à partir d'exemples concrets. En complément de l'étude théorique et pratique, une application web interactive a été développée, offrant une interface conviviale permettant de simuler et de visualiser l'application de ces deux méthodes de clustering sur différents jeux de données. Cette plateforme vise à rendre l'utilisation de ces techniques accessible, même à des utilisateurs non experts en programmation, et à favoriser l'exploration intuitive des résultats.

1.2 Méthodes de Clustering : Une Vue d'Ensemble

Parmi les méthodes de clustering les plus courantes, on retrouve :

K-means : Un algorithme itératif qui cherche à partitionner les données en un nombre prédéfini de clusters en minimisant la somme des distances entre les points de données et les centres des clusters.

Clustering Hiérarchique : Cette méthode organise les données en une hiérarchie de clusters. Elle peut être divisée en deux types : l'agglomératif (fusion des clusters) et le divisif (scission des clusters), permettant ainsi de créer des dendrogrammes pour visualiser les relations entre les clusters.

1.3 Méthodes de Clustering dans l'Analyse de Données

Les méthodes de clustering permettent de regrouper des données similaires en clusters, facilitant ainsi la compréhension des relations et des structures cachées dans des ensembles de données complexes.

Exploration des données non étiquetées : Contrairement aux méthodes supervisées, le clustering ne nécessite pas de données étiquetées, ce qui est particulièrement utile lorsque l'étiquetage est coûteux ou difficile à réaliser.

Applications pratiques :

Analyse de marché : Segmentation des clients en groupes homogènes pour personnaliser les stratégies marketing.

Secteur médical : Groupement de patients ayant des caractéristiques similaires pour identifier des profils de maladies ou affiner des traitements.

Découverte de patterns cachés : Ces méthodes permettent de détecter des structures invisibles à première vue, ouvrant la voie à de nouvelles analyses et à des prises de décisions plus éclairées.

2 Algorithme de K-Means

2.1 Définition et Origines du K-Means

K-Means est un algorithme de clustering non supervisé développé par MacQueen en 1967. Il partitionne un ensemble de données en k clusters en minimisant la variance intra-cluster. L'algorithme commence par initialiser k centroïdes, puis attribue chaque point au centroïde le plus proche et met à jour les centroïdes de manière itérative. C'est une technique permettant d'organiser les données en groupes en fonction de leur similarité. Par exemple, les boutiques en ligne utilisent K-means pour regrouper leurs clients en fonction de leur fréquence d'achat et de leurs dépenses, créant ainsi des segments tels que les acheteurs à petit budget, les acheteurs fréquents et les gros dépensiers pour un marketing personnalisé.

Origines du K-Means : L'algorithme K-means a été introduit en 1957 par Hugo Steinhaus dans un contexte de partitionnement de données. Cependant, il a été popularisé par James MacQueen en 1967, qui a proposé une version plus moderne de l'algorithme. K-means est depuis devenu l'un des algorithmes les plus utilisés pour le clustering en raison de sa simplicité et de son efficacité, notamment pour les données de grande taille.

2.2 Principes Mathématiques du K-Means

Inertie du Cluster

L'inertie I d'un cluster C est définie comme la somme des distances au carré entre chaque point P_i du cluster et son barycentre B , utilisant une certaine norme $\|\cdot\|$. Pour un cluster contenant m points, l'inertie est calculée comme suit :

$$I = \sum_{i=1}^m \|P_i - B\|^2$$

La distance euclidienne

La distance euclidienne $d(P, Q)$ entre deux points $P = (p_1, p_2, \dots, p_n)$ et $Q = (q_1, q_2, \dots, q_n)$ dans un espace n -dimensionnel est donnée par la formule suivante :

$$d(P, Q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

La distance de Manhattan

La distance de Manhattan $d_m(P, Q)$ entre deux points $P = (p_1, p_2, \dots, p_n)$ et $Q = (q_1, q_2, \dots, q_n)$ dans un espace n -dimensionnel est calculée comme suit :

$$d_m(P, Q) = \sum_{i=1}^n |p_i - q_i|$$

Calcul du Barycentre d'un Cluster

Le barycentre B d'un cluster C contenant m points $\{P_1, P_2, \dots, P_m\}$ dans un espace n -dimensionnel est calculé comme suit :

$$B = \left(\frac{\sum_{i=1}^m p_{i1}}{m}, \frac{\sum_{i=1}^m p_{i2}}{m}, \dots, \frac{\sum_{i=1}^m p_{in}}{m} \right)$$

Mise à jour des Centroides

A chaque itération de l'algorithme, les centroids sont mis à jour pour être le centre de gravité des points attribués à chaque cluster. Ce processus est mathématiquement exprimé par :

$$\mu_i = \frac{1}{|S_i|} \sum_{x \in S_i} x$$

Où

- μ_i est le centroid du cluster i
- S_i est l'ensemble des points attribués au cluster i
- $|S_i|$ est le nombre de points dans le cluster i
- x représente un point de données appartenant au cluster i .

Convergence :

L'algorithme continue à itérer entre l'attribution des points aux clusters et la mise à jour des centroids jusqu'à ce que les positions des centroids se stabilisent, c'est-à-dire, les changements dans les positions des centroids entre les itérations successives deviennent négligeables, ou jusqu'à ce qu'un nombre maximum d'itérations soit atteint.

2.3 Fonctionnement de l'Algorithme K-Means

La méthode de k-means ou algorithme de regroupement par centre mobile permet d'effectuer une classification d'un ensemble de données en k clusters. Un cluster regroupe plusieurs concepts similaires. Chaque cluster est décrit par son centre. Les centres des clusters sont mobiles au cours de l'exécution de l'algorithme. **L'algorithme** peut se présenter comme suit:

- définir k ensembles $C_1, \dots, C_k$, initialement vides;
- choisir au hasard k données $d_1, \dots, d_k$;
- pour chaque donnée x :
trouver j dans $1, \dots, k$ qui minimise la distance de x à d_j ;
- ajouter x à C_j ;
- répéter jusqu'à ce que le clustering soit satisfaisant :
- pour j de 1 à k :
 $d_j = \text{barycentre}(C_j)$;

- pour chaque donnée x :
trouver j dans $1, \dots, k$ qui minimise la distance de x à d_j ;
- ajouter x à C_j ;
- renvoyer le clustering $C_1, \dots, C_k$;

La distance euclidienne $d(P, Q)$ entre deux points $P = (p_1, p_2, \dots, p_n)$ et $Q = (q_1, q_2, \dots, q_n)$ dans un espace n -dimensionnel est donnée par la formule suivante :

$$d(P, Q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

2.3.1 Illustration de la Méthode:

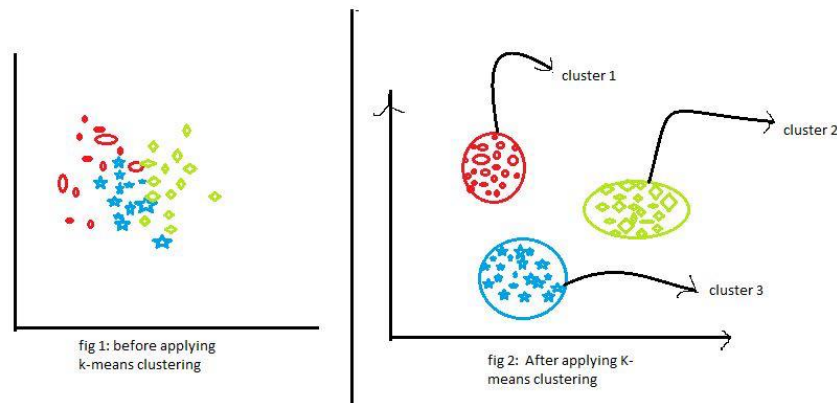


Figure 1: Process K-means

2.3.2 Évaluation de la Performance de l'Algorithme K-Means:

L'évaluation de la performance de l'algorithme de K-Means est importante pour mesurer son efficacité et l'adéquation des clusters formés. Plusieurs métriques sont utilisées pour quantifier la qualité du clustering :

- **Inertie (Somme des Carrés Intra-Cluster)** : L'inertie est définie comme la somme des distances au carré entre les points de données et le centroïde de leur cluster respectif. Mathématiquement, elle est exprimée par :

$$I = \sum_{i=1}^k \sum_{x \in S_i} \|x - \mu_i\|^2$$

où k est le nombre de clusters, S_i est l'ensemble des points dans le cluster i , et μ_i est le centroïde de S_i . Une inertie faible indique que les clusters sont denses et bien séparés.

- **Indice de Silhouette** : Cette métrique évalue à quel point chaque point est proche des points de son propre cluster par rapport aux points des autres clusters. Elle est calculée pour chaque point et est donnée par :

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

où $a(i)$ est la distance moyenne entre i et tous les autres points dans le même cluster, et $b(i)$ est la distance minimale de i à tous les points dans n'importe quel autre cluster dont i n'est pas membre.

- **Score Davies-Bouldin** : Ce score évalue l'efficacité du clustering en combinant la séparation inter-cluster et la compacité intra-cluster. Le score est défini comme la moyenne des ratios les plus élevés de la somme de la dispersion intra-cluster et de la séparation inter-cluster pour chaque cluster :

$$DB = \frac{1}{k} \sum_{i=1}^k \max_{i \neq j} \left(\frac{\sigma_i + \sigma_j}{d(\mu_i, \mu_j)} \right)$$

où σ_i est la dispersion moyenne des points dans le cluster i , μ_i est le centroïde du cluster i , et $d(\mu_i, \mu_j)$ est la distance entre les centroïdes des clusters i et j . Un score plus bas indique un meilleur clustering.

2.4 Avantages et Limites

Simple: Il est facile d'implémenter k-means et d'identifier des groupes de données inconnus à partir d'ensembles de données complexes. Les résultats sont présentés de manière rapide.

Flexible: L'algorithme K-means s'adapte aux divers changements de vos données. En cas de souci, l'ajustement du segment de cluster permettra d'apporter rapidement des modifications nécessaires à l'algorithme.

Convient aux gros data sets: K-means convient à un grand nombre d'ensembles de données et est calculé beaucoup plus rapidement que le plus petit. Il peut également produire des clusters plus élevées.

2.5 Implémentation du K-Means en Python

2.5.1 Environnement de Développement:

Pour l'implémentation de la méthode K-Means, j'ai choisi de combiner **Python** et **Streamlit** comme environnement de développement.

Python a été utilisé pour l'écriture des algorithmes de clustering et la manipulation des données grâce à sa richesse en bibliothèques scientifiques telles que **scikit-learn**, **numpy** et **pandas**.

Streamlit a été choisi pour développer une interface web interactive permettant à l'utilisateur de charger ses données, de configurer les paramètres du clustering, et de visualiser les résultats de manière simple et efficace.

2.5.2 Développement de l'algorithme : Etapes et Code de base :

Étape 1 : Importer les bibliothèques nécessaires

1. **Streamlit** : Utilisé pour créer des interfaces graphiques interactives avec Python.
2. **Pandas** : Utilisé pour la manipulation et l'analyse des données.(Vu importation des données)
3. **NumPy** : Utilisé pour les calculs numériques et les opérations sur les tableaux.
4. **Matplotlib** : Utilisé pour la création de graphiques et de visualisations.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
```

Étape 2 : Créez l'ensemble de données personnalisé avec makeblobs

```
X,y = make_blobs(n_samples = 500,n_features = 2,centers = 3,random_state =
↪ 23)

fig = plt.figure(0)
plt.grid(True)
plt.scatter(X[:,0],X[:,1])
plt.show()
```

Étape 3 : Initialiser les centroïdes aléatoires Le code initialise trois clusters pour le clustering K-means. Il définit une graine aléatoire et génère des centres de cluster aléatoires dans une plage spécifiée, puis crée une liste vide de points pour chaque cluster.

```
k = 3

clusters = {}
np.random.seed(23)

for idx in range(k):
    center = 2*(2*np.random.random((X.shape[1],))-1)
    points = []
    cluster = {
        'center' : center,
        'points' : []
    }

    clusters[idx] = cluster

clusters
```


Étape 4 : Tracez le centre d'initialisation aléatoire avec des points de données

```
plt.scatter(X[:,0],X[:,1])
plt.grid(True)
for i in clusters:
    center = clusters[i]['center']
    plt.scatter(center[0],center[1],marker = '*',c = 'red')
plt.show()
```

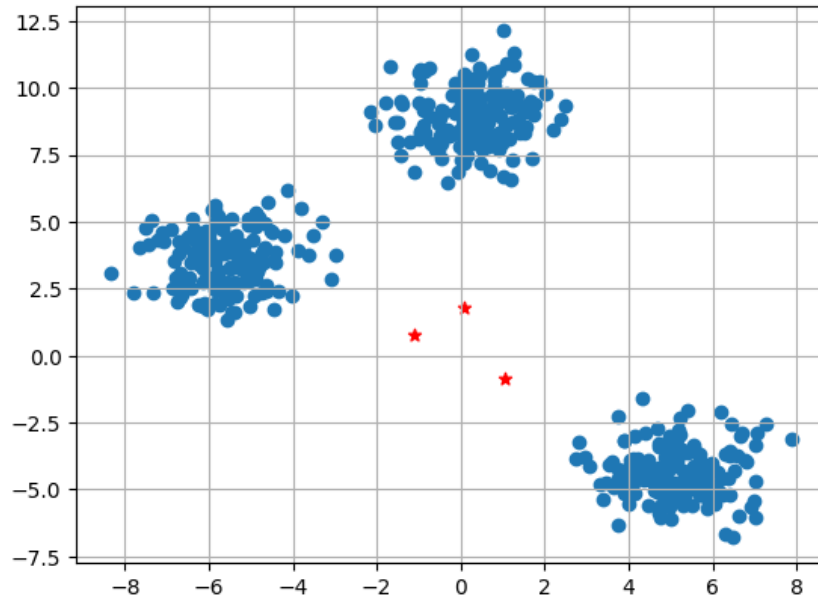


Figure 2: Graphique

La classe `KmeansClustering`

```

import numpy as np
import matplotlib.pyplot as plt
from utils.distances import euclidean_distance, manhattan_distance,
↪ cosine_distance\\
class KmeansClustering:

    def __init__(self, k=3, distance="Euclidean"):
        self.k = k
        self.distance = distance
        self.centroids = None

    def initialize_centroids(self, Data):
        """Initialize centroids as random data points from the dataset."""
        self.centroids = np.random.uniform(np.amin(Data, axis=0), np.amax(Data,
↪ axis=0), size=(self.k, Data.shape[1]))

    def assign_clusters(self, Data):
        """Assign data points to the nearest centroid."""
        clusters = []
        for data_point in Data:
            if self.distance == "Euclidean":
                distances = euclidean_distance(data_point, self.centroids)
            elif self.distance == "Manhattan":
                distances = manhattan_distance(data_point, self.centroids)
            elif self.distance == "Cosine":
                distances = cosine_distance(data_point, self.centroids)
            cluster_num = np.argmin(distances)
            clusters.append(cluster_num)
        return np.array(clusters)

    def update_centroids(self, Data, labels):
        """Update centroids to the mean of assigned data points."""
        new_centroids = np.array([Data[labels == i].mean(axis=0) for i in
↪ range(self.k)])
        return new_centroids

    def fit(self, Data, max_iter=100):
        """Fit the K-means model to the data."""
        self.initialize_centroids(Data)

        for _ in range(max_iter):
            labels = self.assign_clusters(Data)
            new_centroids = self.update_centroids(Data, labels)

            if np.all(np.linalg.norm(self.centroids - new_centroids, axis=1) <
↪ 0.0001):
                break
            else:
                self.centroids = new_centroids

```

```

return labels

def plot_clusters(self, Data, labels):
    """Visualize the clusters and centroids."""
    colors = ["#4C72B0", "#DD8452", "#55A868", "#C44E52", "#8172B3",
              "#937860", "#DA8BC3", "#8C8C8C", "#CCB974", "#64B5CD"]
    fig, ax = plt.subplots()

    for i in range(self.k):
        points = Data[labels == i]
        ax.scatter(points[:, 0], points[:, 1], s=7, c=colors[i %
        ↪ len(colors)], label=f'Cluster {i + 1}')
    ax.scatter(self.centroids[:, 0], self.centroids[:, 1], marker='*',
    ↪ s=120, c='#000000', label='Centroids')
    plt.xlabel('Feature 1')
    plt.ylabel('Feature 2')
    plt.legend()
    plt.show()

    return fig

```

2.5.3 Méthode du Coude pour le Choix de k

La méthode du coude (*Elbow Method*) est une technique utilisée pour déterminer le nombre optimal de clusters k dans l'algorithme K-Means. Elle consiste à exécuter l'algorithme pour différentes valeurs de k et à calculer l'inertie (somme des distances carrées intra-cluster) pour chacune d'elles. L'inertie diminue à mesure que k augmente, mais au-delà d'un certain point, la réduction devient moins significative. Le "coude" de la courbe inertie- k indique la valeur optimale de k , où l'ajout de nouveaux clusters n'apporte plus de gain substantiel en termes de compacité des clusters.

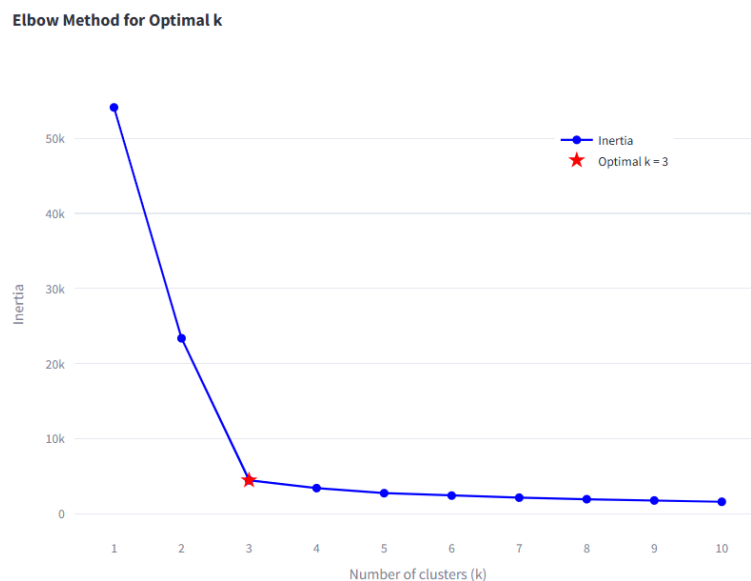


Figure 3: Elbow Method

3 Classification Hiérarchique

3.1 Introduction à la Classification Hiérarchique

La classification hiérarchique est une méthode de regroupement qui construit une hiérarchie de clusters représentée sous forme d'un arbre appelé **dendrogramme**.

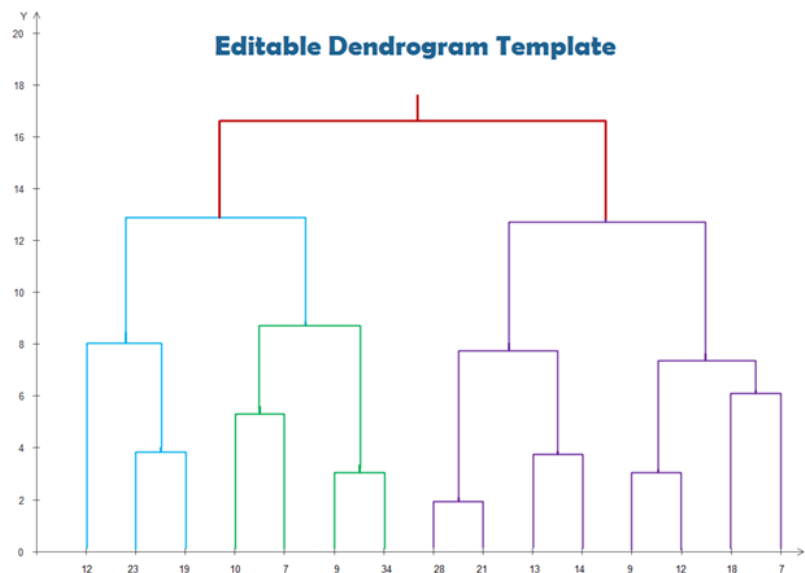


Figure 4: Dendrogramme

Utilité :

Les dendrogrammes aident à choisir le nombre de clusters en visualisant où les fusions créent des augmentations significatives de la dissimilarité, facilitant ainsi une analyse précise et informée des données groupées.

Il existe deux approches principales :

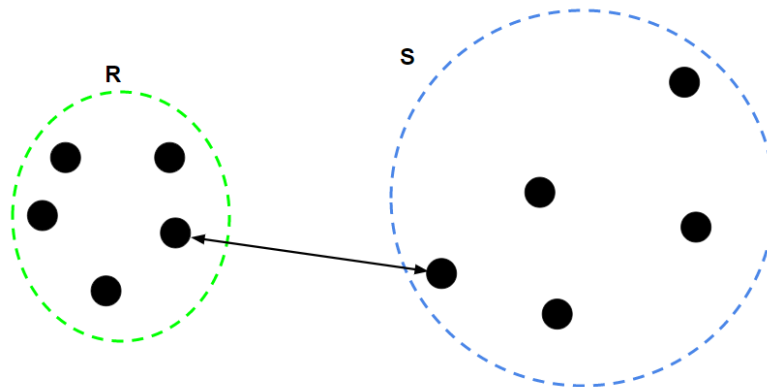
- **Méthode agglomérative (bottom-up)** : Chaque point de données commence comme un cluster individuel. Les clusters les plus proches sont fusionnés progressivement jusqu'à ce qu'il ne reste plus qu'un seul cluster global.
- **Méthode divisive (top-down)** : Tous les points de données commencent dans un même cluster, qui est ensuite divisé de manière réursive en sous-clusters plus petits.

3.2 Mécanique de la Classification Hiérarchique

3.2.1 Méthodes de liaisons :

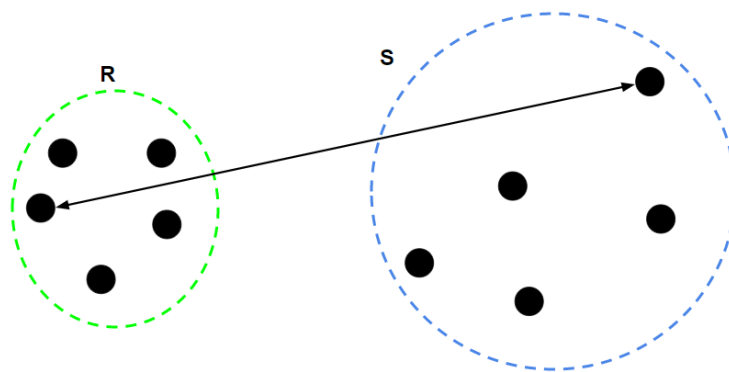
L'algorithme agglomératif commence par considérer chaque point comme un cluster, puis fusionne les paires les plus proches selon des méthodes de liaison (lien simple, complet, moyen). Le processus se répète jusqu'à former un unique cluster.

Lien simple (Single link) : il définit la distance entre deux clusters comme étant la plus petite distances entre toutes les paires de concepts de deux clusters.



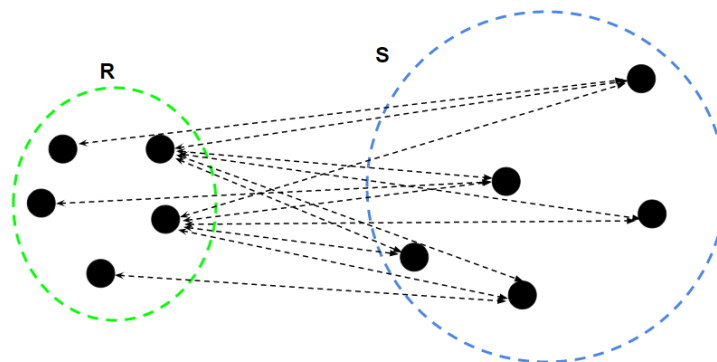
Lien complet (Complete link): Utilise la plus grande distance entre les membres. Elle est définie par :

$$\max\{d(x, y) \mid x \in C_i, y \in C_j\}$$



Lien Moyen de Groupe (Group average link): Calcule la distance moyenne entre tous les membres des deux clusters, donnée par :

$$\frac{1}{|C_i||C_j|} \sum_{x \in C_i, y \in C_j} d(x, y)$$



3.3 Calcul des Distances et Similarités

Les distances euclidienne, Manhattan, Chebyshev, et Minkowski sont utilisées pour mesurer la similarité.

Distance Euclidienne :

$$d(P, Q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

Distance de Manhattan :

La distance de Manhattan mesure la distance en comptant le total des différences verticales et horizontales, donnée par :

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

Distance de Chebychev :

La distance de Chebychev mesure la plus grande différence d'une seule dimension, donnée par :

$$d(x, y) = \max_i |x_i - y_i|$$

Distance de Minkowski :

La distance de Minkowski est une généralisation des distances euclidienne et de Manhattan, donnée par :

$$d(x, y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p}$$

3.4 Critères de Fusion dans la Classification Hiérarchique

Minimisation de la Variance Intra-Cluster : Ce critère vise à minimiser l'augmentation de la variance à l'intérieur des clusters après la fusion, afin de maintenir une homogénéité maximale au sein de chaque groupe. La méthode de Ward est un exemple où ce critère est appliqué. La minimisation de la variance est formulée par :

$$\min \sum_{i \in C} (x_i - \mu_C)^2$$

où C est le cluster résultant de la fusion, x_i sont les points dans C , et μ_C est le centre de gravité (centroid) de C .

Maximisation de la Cohésion : L'objectif est de maximiser la cohésion à l'intérieur des clusters en réduisant la somme des distances entre les points. Ce critère est souvent mesuré par la somme des distances ou des dissimilarités entre les points d'un même cluster. On cherche ainsi à maximiser la fonction suivante :

$$\max \sum_{x, y \in C} -d(x, y)$$

où $d(x, y)$ est la distance entre les points x et y dans le cluster C .

Minimisation de la Connectivité : Ce critère vise à réduire le nombre de liens inter-clusters nécessaires lors de la fusion. Il permet de préserver la structure naturelle des données, en évitant la formation de clusters longs et étendus. La minimisation de la connectivité est formulée comme suit :

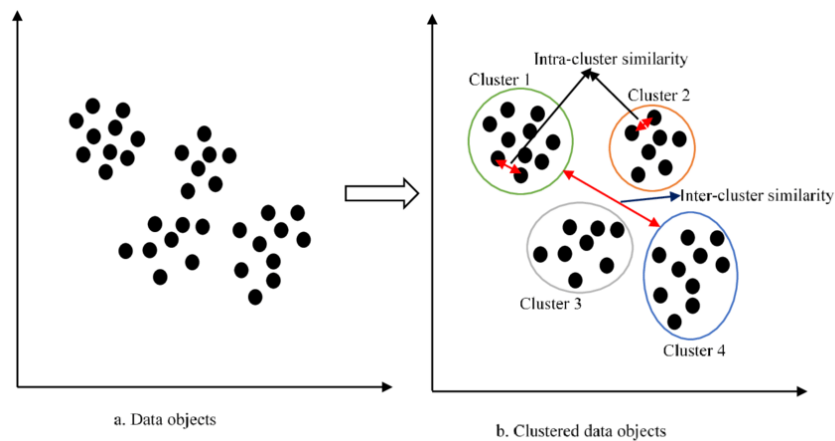
$$\min \sum_{x \in C_1, y \in C_2} d(x, y) \quad \text{tels que} \quad d(x, y) \text{ est minimal pour les clusters } C_1 \text{ et } C_2.$$

Minimisation de la Distance ou Dissimilarité : Ce critère consiste à minimiser la distance ou la dissimilarité spécifiée entre les clusters, selon la méthode de liaison choisie (simple, complète, moyenne, etc.). Cette minimisation est exprimée par :

$$\min D(C_1, C_2)$$

où D est la mesure de distance ou dissimilarité entre les clusters C_1 et C_2 .

Ces critères influencent directement la formation des clusters ainsi que leur interprétation, selon les propriétés et la structure des données analysées.



3.5 Implémentation de CAH en Python

3.5.1 Choix de l'environnement pour CAH :

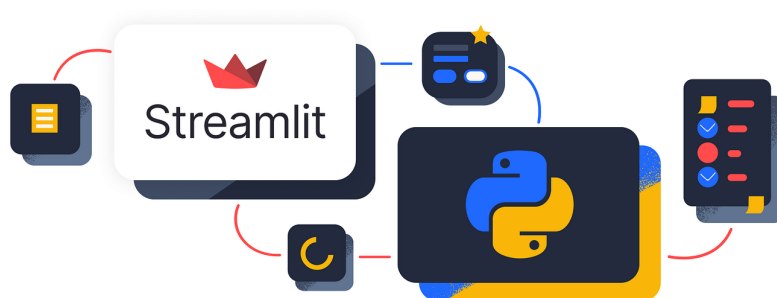
Pour l'implémentation de la méthode de classification hiérarchique (CAH), le langage de programmation Python a été choisi en raison de ses puissantes bibliothèques pour le traitement des données et l'apprentissage automatique. Python est largement utilisé dans le domaine de l'analyse de données, et plusieurs de ses bibliothèques offrent des outils optimisés pour la mise en œuvre des algorithmes de clustering.

Bibliothèques utilisées :

- **Scikit-learn** : Cette bibliothèque fournit une implémentation efficace des méthodes de clustering, y compris la méthode de clustering hiérarchique. Elle offre des fonctions pour l'agrégation des clusters, le calcul des distances, et la visualisation des résultats.
- **SciPy** : Utilisée pour les calculs de distance et les méthodes de linkage, elle permet de gérer les différentes étapes du processus de clustering hiérarchique de manière optimisée.
- **Matplotlib et Seaborn** : Ces bibliothèques sont employées pour la visualisation des résultats, telles que la création de dendrogrammes et de graphiques représentant les clusters.

Streamlit pour l'interface web : Pour rendre l'implémentation plus accessible et interactive, l'interface de l'application est développée avec **Streamlit**. Cette bibliothèque permet de créer facilement des applications web où l'utilisateur peut charger ses données, ajuster les paramètres de l'algorithme, et voir les résultats en temps réel sous forme de visualisations interactives.

Pourquoi cet environnement ? : Ce choix d'environnement permet de combiner la puissance de Python pour l'analyse de données avec l'interactivité de Streamlit pour rendre les résultats plus accessibles. De plus, Python permet d'exécuter des algorithmes complexes tout en offrant une grande flexibilité pour l'extension de l'application à d'autres méthodes de clustering.



La classe hierarchicalClustering :

```
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt

class hierarchicalClustering:

    def __init__(self, n_clusters=3, linkage='ward'):
        self.n_clusters = n_clusters
        self.linkage = linkage
        self.model = AgglomerativeClustering(n_clusters=n_clusters,
        ↪ linkage=linkage)

    def fit_predict(self, Data):
        self.labels = self.model.fit_predict(Data)
        self.linked = linkage(Data, method=self.linkage)
        return self.labels

    def plot_dendrogram(self):
        fig, ax = plt.subplots(figsize=(10, 7))
        dendrogram(self.linked, orientation='top', distance_sort='descending',
        ↪ show_leaf_counts=True, ax=ax)
        ax.set_title('Hierarchical Clustering Dendrogram')
        ax.set_xlabel('Sample Index')
        ax.set_ylabel('Distance')
        return fig

    def plot_clusters(self, Data, labels):
        fig, ax = plt.subplots()
        scatter = ax.scatter(Data[:, 0], Data[:, 1], c=labels, cmap='viridis',
        ↪ label='Data Points')
        ax.set_title('Hierarchical Clustering Results')
        ax.set_xlabel('Feature 1')
        ax.set_ylabel('Feature 2')
        ax.legend()
        return fig
```

Explication Du Code :

Pour implémenter la méthode de clustering hiérarchique, une classe `hierarchicalClustering` a été définie en Python, utilisant les bibliothèques `Scikit-learn` et `SciPy`. Cette classe permet de réaliser le clustering hiérarchique agglomératif (Agglomerative Clustering) et de visualiser les résultats sous forme de dendrogramme et de diagramme de dispersion des clusters.

Structure de la classe : La classe `hierarchicalClustering` possède plusieurs méthodes pour effectuer l'analyse et visualiser les résultats. Les principales parties de cette classe sont :

- **Initialisation (`__init__`) :** La méthode `__init__` initialise les paramètres de l'algorithme, à savoir le nombre de clusters (`n_clusters`) et la méthode de liaison (`linkage`). Le modèle de clustering hiérarchique est ensuite créé avec `AgglomerativeClustering`

de la bibliothèque `Scikit-learn`, en utilisant ces paramètres.

- **Entraînement et prédiction (`fit_predict`)** : La méthode `fit_predict` applique l'algorithme de clustering sur les données d'entrée. Elle calcule également le lien hiérarchique des données à l'aide de la fonction `linkage` de la bibliothèque `SciPy`. Cette fonction produit un tableau de liens pour la création du dendrogramme.
- **Dendrogramme (`plot_dendrogram`)** : La méthode `plot_dendrogram` génère un dendrogramme représentant l'arbre hiérarchique du clustering. Le dendrogramme est tracé à l'aide de la fonction `dendrogram` de `SciPy` et est personnalisé avec un titre, ainsi que des étiquettes pour les axes.
- **Visualisation des clusters (`plot_clusters`)** : La méthode `plot_clusters` permet de visualiser les clusters formés par l'algorithme sur un graphique 2D. Les points de données sont affichés sur le plan avec des couleurs distinctes pour chaque cluster. Le graphique est généré à l'aide de `Matplotlib`.

4 Résultats : Application Web de Clustering

4.1 Code Source de l'Application

```
import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import plotly.express as px
import plotly.graph_objects as go
from sklearn.cluster import KMeans
from kmeans_Clustering import KmeansClustering
from hierarchical_Clustering import hierarchicalClustering
from style import styles

styles()

# Implémentation améliorée d'ElbowMethod
class ElbowMethod:
    def __init__(self, data, k_range=(1, 10)):
        self.data = data
        self.k_min, self.k_max = k_range
        self.inertia_values = []
        self.k_values = list(range(self.k_min, self.k_max + 1))

    def fit(self):
        """Calculate inertia for different k values."""
        self.inertia_values = []
        for k in self.k_values:
            if k <= 1: # KMeans special case for k=1
                if k == 0:
                    self.inertia_values.append(float('inf'))
                else:
                    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
                    kmeans.fit(self.data)
                    self.inertia_values.append(kmeans.inertia_)
            continue

            kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
            kmeans.fit(self.data)
            self.inertia_values.append(kmeans.inertia_)

        return self

    def find_optimal_k(self):
        """Find the optimal k using the elbow method."""
        if not self.inertia_values:
            self.fit()

        if len(self.inertia_values) <= 2:
            return 2 # Default if not enough points
```

```

# Simple elbow detection - find the k where the rate of decrease
↳ sharply changes
inertia_diffs = np.diff(self.inertia_values)
inertia_diffs2 = np.diff(inertia_diffs)
optimal_idx = np.argmax(inertia_diffs2) + 1 # +1 because of double
↳ diff

# Ensure the index is within valid range
if optimal_idx >= len(self.k_values):
    optimal_idx = len(self.k_values) - 1

return self.k_values[optimal_idx]

def plot_elbow_curve(self):
    """Plot the elbow curve using Plotly."""
    if not self.inertia_values:
        self.fit()

    optimal_k = self.find_optimal_k()
    optimal_idx = self.k_values.index(optimal_k)

    # Create Plotly figure
    fig = go.Figure()
    fig.add_trace(go.Scatter(
        x=self.k_values,
        y=self.inertia_values,
        mode='lines+markers',
        name='Inertia',
        marker=dict(size=8, color='blue')
    ))

    # Add point for optimal k
    fig.add_trace(go.Scatter(
        x=[optimal_k],
        y=[self.inertia_values[optimal_idx]],
        mode='markers',
        name=f'Optimal k = {optimal_k}',
        marker=dict(size=12, color='red', symbol='star')
    ))

    # Update layout
    fig.update_layout(
        title='Elbow Method for Optimal k',
        xaxis=dict(title='Number of clusters (k)', tickmode='linear'),
        yaxis=dict(title='Inertia'),
        legend=dict(x=0.7, y=0.9),
        hovermode='closest',
        height=600
    )

    return fig

```

```

# Functions for Plotly visualizations
def plot_clusters_2d_plotly(data, labels, title="2D Cluster Visualization"):
    """Plot clusters in 2D using Plotly."""
    if data.shape[1] < 2:
        st.error("Need at least 2 dimensions for 2D visualization.")
        return None

    df = pd.DataFrame(data[:, :2], columns=['Dimension 1', 'Dimension 2'])
    df['Cluster'] = labels

    fig = px.scatter(
        df, x='Dimension 1', y='Dimension 2',
        color='Cluster',
        color_continuous_scale=px.colors.qualitative.G10,
        title=title,
        height=600
    )

    fig.update_layout(
        legend_title="Cluster",
        coloraxis_colorbar=dict(title="Cluster")
    )

    return fig

def plot_clusters_3d_plotly(data, labels, title="3D Cluster Visualization"):
    """Plot clusters in 3D using Plotly."""
    if data.shape[1] < 3:
        st.error("Need at least 3 dimensions for 3D visualization.")
        return None

    df = pd.DataFrame(data[:, :3], columns=['Dimension 1', 'Dimension 2',
        ↪ 'Dimension 3'])
    df['Cluster'] = labels

    fig = px.scatter_3d(
        df, x='Dimension 1', y='Dimension 2', z='Dimension 3',
        color='Cluster',
        color_continuous_scale=px.colors.qualitative.G10,
        title=title,
        height=700
    )

    fig.update_layout(
        scene=dict(
            xaxis_title='Dimension 1',
            yaxis_title='Dimension 2',
            zaxis_title='Dimension 3'
        ),
        legend_title="Cluster",
        coloraxis_colorbar=dict(title="Cluster")
    )

    return fig

```

```

def plot_dendrogram_plotly(Z, labels=None,
↪ title="Hierarchical Clustering Dendrogram"):
    """Plot dendrogram using Plotly."""
    import scipy.cluster.hierarchy as sch

    # Create dendrogram figure
    fig = go.Figure()

    # Create dendrogram trace
    dendro = sch.dendrogram(Z, no_plot=True)

    # Extract coordinates
    x = []
    y = []
    for i, d in enumerate(dendro['dcoord']):
        x.extend([dendro['icoord'][i][j] for j in range(len(d))])
        x.append(None)
        y.extend(d)
        y.append(None)

    # Add trace
    fig.add_trace(go.Scatter(
        x=x,
        y=y,
        mode='lines',
        line=dict(color='black', width=1.5),
        hoverinfo='none'
    ))

    # Update layout
    fig.update_layout(
        title=title,
        xaxis=dict(
            showline=False,
            showgrid=False,
            zeroline=False,
            showticklabels=False
        ),
        yaxis=dict(
            title='Distance',
            showgrid=False,
            zeroline=False
        ),
        height=600,
        margin=dict(l=40, r=40, b=40, t=80)
    )

    return fig

```

```

# Function Definitions
def run_kmeans_clustering(data, n_clusters, distance_metric, use_3d=False):
    if data.shape[1] < 2:
        ↪ st.error("The dataset must contain at least two numeric columns for clustering.")
        return

    kmeans = KmeansClustering(k=n_clusters, distance=distance_metric)
    labels = kmeans.fit(data.values)

    if use_3d and data.shape[1] >= 3:
        fig = plot_clusters_3d_plotly(data.values, labels,
        ↪ f"K-Means Clustering (K={n_clusters}) - 3D View")
        st.plotly_chart(fig, use_container_width=True)
    else:
        # Check if KmeansClustering.plot_clusters returns a matplotlib figure
        try:
            # Try original method first
            fig = kmeans.plot_clusters(data.values, labels)
            st.pyplot(fig)
        except Exception:
            # Fallback to Plotly visualization
            fig = plot_clusters_2d_plotly(data.values, labels,
            ↪ f"K-Means Clustering (K={n_clusters}) - 2D View")
            st.plotly_chart(fig, use_container_width=True)

def run_hierarchical_clustering(data, n_clusters_hier, linkage, use_3d=False):
    if data.shape[1] < 2:
        ↪ st.error("The dataset must contain at least two numeric columns for clustering.")
        return

    hierarchical_clustering =
    ↪ hierarchicalClustering(n_clusters=n_clusters_hier, linkage=linkage)
    labels = hierarchical_clustering.fit_predict(data.values)

    # For dendrogram, try to get the linkage matrix if available
    try:
        Z = hierarchical_clustering.Z # Assuming Z is stored as an attribute
        fig_dendrogram = plot_dendrogram_plotly(Z,
        ↪ title="Hierarchical Clustering Dendrogram")
        st.plotly_chart(fig_dendrogram, use_container_width=True)
    except:
        # Fallback to the original method
        try:
            fig_dendrogram = hierarchical_clustering.plot_dendrogram()
            st.pyplot(fig_dendrogram)
        except Exception as e:
            st.error(f"Could not plot dendrogram: {e}")

```

```

if use_3d and data.shape[1] >= 3:
    fig = plot_clusters_3d_plotly(data.values, labels,
    ↪ f"Hierarchical Clustering (K={n_clusters_hier}) - 3D View")
    st.plotly_chart(fig, use_container_width=True)
else:
    # Try original method first
    try:
        fig_clusters = hierarchical_clustering.plot_clusters(data.values,
        ↪ labels)
        st.pyplot(fig_clusters)
    except:
        # Fallback to Plotly visualization
        fig = plot_clusters_2d_plotly(data.values, labels,
        ↪ f"Hierarchical Clustering (K={n_clusters_hier}) - 2D View")
        st.plotly_chart(fig, use_container_width=True)

def run_Elbow_Method(data):
    try:
        elbow = ElbowMethod(data, k_range=(1, 10))
        elbow.fit()
        optimal_k = elbow.find_optimal_k()
        fig = elbow.plot_elbow_curve()

        ↪ st.write("La méthode du coude est une heuristique utilisée pour déterminer le nombre
        st.plotly_chart(fig, use_container_width=True)
        st.write(f"Nombre optimal de clusters : {optimal_k}")
        return optimal_k
    except AttributeError as e:
        st.error(f"Erreur dans ElbowMethod : {e}")
        st.write("Utilisation de la valeur par défaut de 3 clusters.")
        return 3

def show_kmeans_info():
    """Display information about K-means clustering."""
    st.markdown("""
    ## K-Means Clustering

    ### Description
    K-Means est un algorithme de clustering par partitionnement qui divise un ensemble de données

    ### Fonctionnement
    1. **Initialisation**: Sélection aléatoire de K centroïdes parmi les données
    2. **Attribution**: Affectation de chaque point au centroïde le plus proche
    3. **Mise à jour**: Recalculer la position des centroïdes en prenant la moyenne des points as
    4. **Répétition**: Répéter les étapes 2 et 3 jusqu'à convergence

    ### Avantages
    - Simple à comprendre et à implémenter
    - Efficace et rapide pour les grands ensembles de données
    - Fonctionne bien pour des clusters de forme sphérique

    ### Limites
    - Nécessite de spécifier le nombre K de clusters à l'avance
    - Sensible à l'initialisation des centroïdes
    - Difficulté à gérer des clusters de formes irrégulières
    - Sensibilité aux valeurs aberrantes

    ### Métriques de distance
    - **Euclidienne**: Distance en ligne droite entre deux points
    - **Manhattan**: Somme des différences absolues des coordonnées
    - **Cosine**: Mesure l'angle entre deux vecteurs (similarité)
    """)

```



```

def show_hierarchical_info():
    """Display information about hierarchical clustering."""
    st.markdown("""
    ## Classification Hiérarchique

    ### Description
    La classification hiérarchique construit une hiérarchie de clusters, généralement représentée

    ### Fonctionnement
    1. **Approche agglomérative (bottom-up)**:
        - Commencer avec chaque point comme cluster distinct
        - Fusionner progressivement les clusters les plus proches
        - Continuer jusqu'à n'avoir plus qu'un seul cluster

    2. **Approche divisive (top-down)**:
        - Commencer avec un seul cluster contenant tous les points
        - Diviser récursivement les clusters
        - S'arrêter quand chaque cluster ne contient qu'un seul point

    ### Méthodes de liaison
    - **Single linkage**: Distance entre les points les plus proches de deux clusters
    - **Complete linkage**: Distance entre les points les plus éloignés
    - **Average linkage**: Moyenne des distances entre tous les points
    - **Ward**: Minimise la variance intra-cluster

    ### Avantages
    - Pas besoin de spécifier le nombre de clusters à l'avance
    - Fournit une représentation hiérarchique intuitive (dendrogramme)
    - Peut découvrir des clusters de différentes formes et tailles

    ### Limites
    - Complexité computationnelle élevée ( $O(n^2)$  ou plus)
    - Sensibilité au bruit et aux valeurs aberrantes
    - Difficile à appliquer sur de très grands ensembles de données
    """)

# Streamlit App Layout
st.title('Clustering Visualization')
st.write("by [Loubna Tildi](https://www.linkedin.com/in/loubna-tildi-4ba51a285/?locale=en_US)")

# Sidebar for clustering options
method = st.sidebar.radio("Choose a clustering method:", ('K-Means Clustering',
↳ 'Hierarchical Clustering'), key='clustering_method')

# Add information section in sidebar
with st.sidebar.expander("Information sur les méthodes"):
    info_option = st.radio(
        "Sélectionnez une méthode pour en savoir plus:",
        ["K-Means", "Classification Hiérarchique"]
    )

# Information display based on selection
if info_option == "K-Means":
    show_kmeans_info()
elif info_option == "Classification Hiérarchique":
    show_hierarchical_info()

```

```

# Add 3D visualization option
use_3d_viz = st.sidebar.checkbox("Utiliser la visualisation 3D", False,
    ↪ key='use_3d_viz')
if use_3d_viz and 'data' in st.session_state:
    if st.session_state.data.shape[1] < 3:
        ↪ st.sidebar.warning("La visualisation 3D nécessite au moins 3 dimensions. Utilisez un
        use_3d_viz = False

n_clusters = None
distance_metric = None
linkage = "ward" # Default linkage method

if method == 'K-Means Clustering':
    distance_metric = st.sidebar.selectbox('Distance metric', ('Euclidean',
    ↪ 'Manhattan', 'Cosine'), key='kmeans_distance_metric')
    use_optimal_k = st.sidebar.checkbox("Use optimal K from Elbow Method",
    ↪ True, key='use_optimal_k')
    if use_optimal_k:
        if 'optimal_k' not in st.session_state:
            st.session_state.optimal_k = 3
        n_clusters = st.session_state.optimal_k
    else:
        n_clusters = st.sidebar.slider('Number of clusters', value=3,
        ↪ min_value=2, max_value=10, step=1, key='kmeans_n_clusters')

n_clusters_hier = None
if method == 'Hierarchical Clustering':
    linkage = st.sidebar.selectbox('Linkage method', ('ward', 'single',
    ↪ 'complete', 'average'), key='hier_linkage')
    n_clusters_hier = st.sidebar.slider('Number of clusters', value=3,
    ↪ min_value=2, max_value=10, step=1, key='hier_n_clusters')

# Data input section
st.write("### Data Input:")
data_input_method = st.radio("Choose how to input data:", ('Upload a file',
    ↪ 'Manual input', 'Random dataset'), key='data_input_method')

if data_input_method == 'Upload a file':
    uploaded_file = st.file_uploader("Choose a file", type=['csv', 'txt'])
    if uploaded_file is not None:
        st.session_state.data = pd.read_csv(uploaded_file)
elif data_input_method == 'Manual input':
    num_columns = st.number_input("Number of columns", min_value=1, value=3 if
    ↪ use_3d_viz else 2, step=1)
    column_names = [st.text_input(f"Column {i+1} name:",
    ↪ value=f"Feature {i+1}", key=f'manual_col_name_{i}') for i in
    ↪ range(num_columns)]
    num_rows = st.number_input("Number of rows to generate", min_value=1,
    ↪ value=100, step=1)
    min_val, max_val = st.slider("Value range for random data", 0, 100, (0,
    ↪ 100))

    if st.button("Generate Data"):
        random_data = np.random.randint(min_val, max_val+1, size=(num_rows,
        ↪ len(column_names)))
        st.session_state.data = pd.DataFrame(random_data, columns=column_names)
elif data_input_method == 'Random dataset':
    st.session_state.data = pd.read_csv('datasets/data1.txt', header=None)

```

```
# Ensure data preview and clustering execution only if data is available
if 'data' in st.session_state and not st.session_state.data.empty:
    st.write("#### Data Preview:")
    st.write(st.session_state.data)

# Preprocess data
numeric_data = st.session_state.data.apply(pd.to_numeric,
    ↪ errors='coerce').dropna()

# Clustering execution
if method == 'K-Means Clustering' and n_clusters and distance_metric:
    st.write("### Choosing the best K parameter for K-Means Clustering:")
    st.session_state.optimal_k = run_Elbow_Method(numeric_data)
    st.write("### K-Means Clustering:")
    run_kmeans_clustering(numeric_data, n_clusters, distance_metric,
    ↪ use_3d=use_3d_viz)

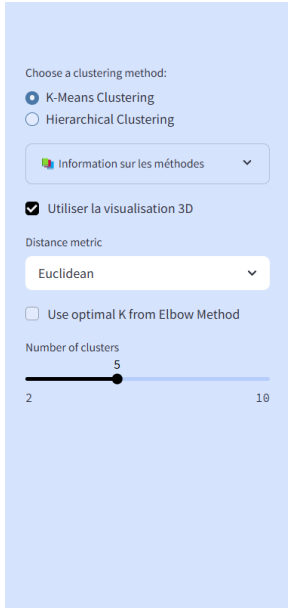
elif method == 'Hierarchical Clustering' and n_clusters_hier:
    run_hierarchical_clustering(numeric_data, n_clusters_hier, linkage,
    ↪ use_3d=use_3d_viz)
```

4.2 L'Application Web

L'application web, développée avec Streamlit, permet aux utilisateurs d'importer des données, de sélectionner le nombre de clusters, et de visualiser les résultats en 2D et 3D pour K-Means, ainsi que le dendrogramme pour la classification hiérarchique.

Lien de l'Application Web : <http://localhost:8503>

4.3 Visualisation de l'Application Web



Analyse de Données par Clustering : K-Means et CAH

by [Loubna Tildi](#)

K-Means Clustering

Description

K-Means est un algorithme de clustering par partitionnement qui divise un ensemble de données en K groupes distincts en minimisant la variance intra-cluster.

Fonctionnement

1. **Initialisation:** Sélection aléatoire de K centroïdes parmi les données
2. **Attribution:** Affectation de chaque point au centroïde le plus proche
3. **Mise à jour:** Recalculer la position des centroïdes en prenant la moyenne des points assignés
4. **Répétition:** Répéter les étapes 2 et 3 jusqu'à convergence

Figure 5: Interface principale de l'application de clustering

K-Means Clustering:



K-Means Clustering (K=6) - 3D View

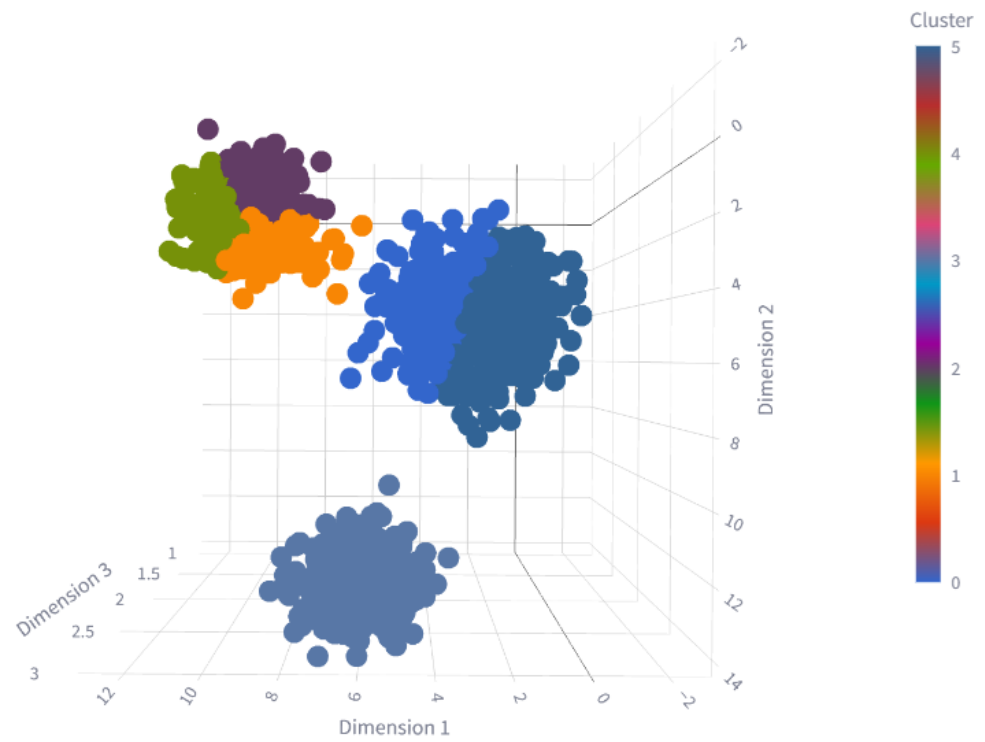


Figure 6: Clustering par la méthode K-means sur un plan 3D

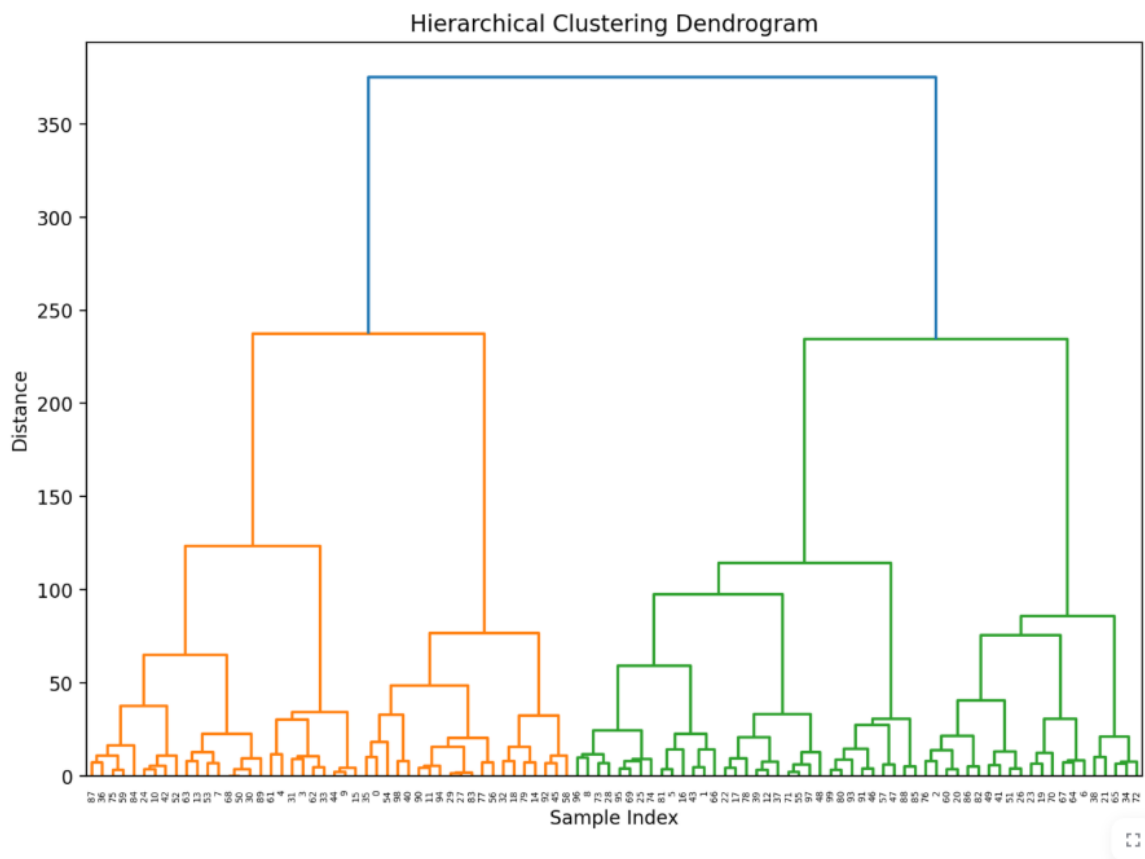


Figure 7: Dendrogramme obtenu par la méthode CAH

Conclusion

Ce rapport a consisté à étudier et mettre en œuvre deux méthodes majeures de clustering: K-Means et la Classification Ascendante Hiérarchique (CAH). Après une présentation théorique des concepts et des algorithmes, leur implémentation a été réalisée en Python, accompagnée d'une interface web développée avec Streamlit. L'application conçue permet de charger des jeux de données, d'appliquer les méthodes de clustering, et de visualiser les résultats à travers des graphes interactifs tels que les nuages de points et les dendrogrammes. Ce travail montre ainsi l'efficacité des techniques de clustering dans la structuration automatique de données complexes, et leur pertinence dans des applications réelles d'aide à la décision.